Storage Organization

Run time storage:

- The compiler demands for a block of memory to OS. The compiler utilizes the block of memory for running the compiled program. This block of memory is called run time storage.

⟹ The run time storage is subdivided to hold code and data such as:

- The generated target code
- Data objects
- Information which keeps track of procedure activations.

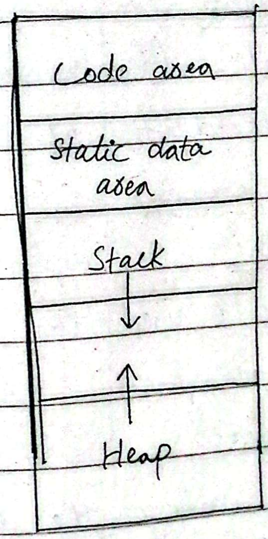
⟹ The size of generated code is fixed.

⇒ Hence the target code occupies the statistically determined area of the memory.

⇒ Compiler places the target code at the lower end of the memory.

⇒ The amount of memory required by the data objects is known at the compiled time.

⇒ Can also be placed in statistically allocated area.



Run time storage organization

Date:

Day: M T W T F

Storage Allocation Strategies

Run time storage is divided as:

- Code Area
- Static data area
- Stack area
- Heap area

⟹ There are different storage allocation strategies based on the division of run time storage:

Static allocation:

- The static allocation is for all the data objects at compile time.

Stack allocation:

- A stack is used to manage run time environment.

Heap allocation:

- Used to manage the dynamic memory allocation.

Date:

Day: M T W T F S

Comparison between static, stack and Heap Allocation:

Static allocation	Stack allocation	Heap allocation
Storage allocation is done for all data objects at compile time.	Stack is used to manage run-time Environment.	Heap is used to manage the dynamic memory allocation
Data structure can not be created dynamically because in static allocation compiler can determine the amount the of storage required by each data object.	Data structures and data objects can be created dynamically.	Data structures and data objects can be created dynamically.

Memory Allocation: ○ The names of data objects are bound to storage at compile time.	○ Using LIFO activation records and data objects are pushed on to the stack. The memory addressing can be done using index and registers.	○ A contiguous block of memory from heap is allocated for activation record or data object. A linked list is maintained for free blocks
Merits and Limitations: ○ simple to implement ○ Supports static allocation ○ Recursively Procedures are not supported	Supports ○ dynamic memory allocation. ○ Slower ○ Support recursive Procedure ○ References to ~~non-local variable~~ are not retained.	○ Efficient memory management using linked list. ○ Deallocated space can be reused. ○ Hole may be get introduced in this scheme

Q) Explain Activation Record?

Definitions:

⟹ The activation record is a block of memory used for managing information needed by a single execution of a procedure.

Various fields of Activation record:

- ○ Temporary values ○ Local variables
- ○ saved machine registers ○ Control link
- ○ Access link ○ Actual parameters
- ○ Return values

Return value
Actual Parameters
Control link (Dynamic link)
Access link (Static link)
Saved machine status
Local variables
Temporaries

Date:

Day: M T W T F S

Q. What is activation tree?

⟹ Data structure used for specifying the flow of control among the procedures.

⟹ In an activation tree:-

(1) Each node shows an activation of procedure.

(2) For showing activation of main program root is used.

(3) if lifetime of 'x' comes before the lifetime of 'y' then x becomes left child of y.

(4) if control flows from x to y then 'x' is parent node of 'y'.

Q. Enlist the storage allocation strategies in run time environment?

(1) Static allocation

(2) Stack allocation.

(3) Heap allocation

Date:

Day: M T W T F S

Q. What is the purpose of control stack used in run time storage organization?

o The control stack is used to manage the active procedures.

Managing the active procedures means that when a call occurs then the execution of activation is interrupted and the information about the status of the control flow is used.

Q. Suggest a suitable approach for computing the hash function?

o Using hash function we should obtain exact locations of name in symbol table.

o The hash function should be such that there will be minimum number of collisions.

Q. Why hash tables are implemented?

design for efficient symbol table management.
- provides an effective way to perform symbol table operations such as lookups & insertion.

o Hashing is important procedure used to search the records of the symbol table.

The hash table consists of K entries from 0 to K-1.

These entries are a pointer to the symbol table pointing to the names of the symbol table.

It can decide whether "Name" is in the symbol table we use a hash function h will result an integer.

- support for dynamic scope management, flexibility in adapting to language characteristics.

Q. What is implicit deallocation?

o Implicit deallocation or garbage collection is the automatic reallocation of memory that is no longer in use by the application program.

o Implicit deallocation requires the cooperation between user program & runtime packages. This is implemented by fixing the format of storage blocks.

Optional blocksize
Optional reference count
Optional mark
Pointers to blocks
User information

- refers to automatic release of memory or resources without explicit instruction from the programmer
- crucial for managing memory efficiently and preventing memory leaks in programs.
- ID is often associated with languages that have automatic memory management

⇒ Two approaches can be used for implicit deallocation: garbage collection in languages such as java, Python

o Reference Counts
o Masking techniques.

Q. Write techniques needed to implement dynamic storage allocation?

⇒ The techniques needed to implement dynamic storage allocation techniques depends on how the space is deallocated. ie explicitly & implicitly

Date:
Day: M T W T F S

- Explicit allocation of fixed size block
- Explicit allocation of variable size block
- Implicit deallocation

Malloc & Free
calloc & Relloc
memory pools
Buddy Allocation
Garbage collection
Stack Allocation
Reference Counting
First Fit, Best Fit, Worst Fit